



AETHERION
— INDEPENDENT REVIEW —

INDEPENDENT SECURITY REVIEW

PasswordStore

Author: Venkatesh Pamarthi
github.com/venkateshpamarthi

Aetherion is only a personal mark on the page — a small signature, not a firm, studio, or client engagement.

PROTOCOL	PasswordStore
----------	---------------

REVIEW TYPE	Educational / portfolio review
-------------	--------------------------------

VERSION	0.1
---------	-----

DATE	27 September 2026
------	-------------------

COMMIT	2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
--------	--

COMPILER	Solidity 0.8.18
----------	-----------------

CHAIN	Ethereum
-------	----------

IN SCOPE	src/PasswordStore.sol
----------	-----------------------

TABLE OF CONTENTS

1. About
2. Disclaimer
3. Risk classification
4. Audit details
5. Protocol summary
6. Executive summary
7. Findings

ABOUT

Venkatesh Pamarthi is an independent smart contract reviewer and former blockchain developer. This document is an educational portfolio review of PasswordStore. It is not a client engagement. Aetherion is only a personal mark on the page.

DISCLAIMER

I made a best effort to find as many issues as I could in the time boxed for this review. I do not claim the list is complete, and I take no responsibility for how these findings are used. A security review is not an endorsement of the protocol or the product. The work here is limited to the security of the Solidity implementation in scope.

RISK CLASSIFICATION

Issues are graded by impact and likelihood.

		IMPACT		
		HIGH	MEDIUM	LOW
LIKELIHOOD	HIGH	H	H/M	M
	MEDIUM	H/M	M	M/L
	LOW	M	M/L	L

- **High impact:** assets lost, confidentiality of the stored password broken, or the owner loses control of the store.
- **Medium impact:** a subset of users or a recoverable failure of a core path.
- **Low impact:** unexpected behavior with limited or no direct loss.
- **Informational:** documentation, clarity, or best-practice notes with no runtime loss.

AUDIT DETAILS

Findings in this document correspond to commit:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

SCOPE

```
src/  
└─ PasswordStore.sol
```

Out of scope: deployment scripts, tests, and dependencies except where they help prove a finding.

PROTOCOL SUMMARY

PasswordStore is a single-contract application for storing and retrieving one user’s password. The design is for a single owner. Only that owner should be able to set or read the password, and others should not be able to see it.

ROLES

- **Owner:** the deployer (`msg.sender` in the constructor). Only this address should set or read the password. For this contract, only the owner should interact with it.

EXECUTIVE SUMMARY

This review covers `PasswordStore.sol` only. It is not a paid audit. The contract cannot keep a password secret on-chain, and `setPassword` has no owner check. Those two issues break the stated product. A missing constructor password and incorrect NatSpec are smaller.

SEVERITY	COUNT
High	2
Medium	0
Low	1
Informational	1
Gas	0
Total	4

ID	FINDING	SEVERITY
H-1	On-chain password storage is publicly readable regardless of the <code>private</code> visibility specifier	HIGH
H-2	Missing access control on <code>setPassword</code> allows anyone to overwrite the stored password	HIGH
L-1	The constructor does not set an initial password, so the store is empty until the first <code>setPassword</code>	LOW
I-1	Incorrect NatSpec on <code>getPassword</code> documents a parameter the function does not accept	INFORMATIONAL

FINDINGS

[H-1] On-chain password storage is publicly readable regardless of the `private` visibility specifier

DESCRIPTION

`PasswordStore::s_password` is declared `private` and is only meant to be returned to the owner through `PasswordStore::getPassword`. That visibility specifier does **not** keep the value secret.

On Ethereum, every contract storage slot is public. `private` only prevents other *contracts* from reading the variable in Solidity. Anyone can still read the slot directly from a node, a block explorer, or tools such as `cast storage`.

The protocol's stated goal is that "others won't be able to see" the password. Storing the plaintext password on-chain breaks that guarantee, even if `getPassword` correctly restricts callers.

```
address private s_owner;  
// @audit storage is public on-chain; `private` does not hide this value  
string private s_password;
```

IMPACT

Any observer can recover the stored password. This breaks the core product promise of a private password store and fully compromises confidentiality.

PROOF OF CONCEPT

1. Start a local node: `anvil`
2. Deploy the contract: `make deploy`
3. Read storage slot `1` (`s_password`). On a default Anvil deployment the contract address is `0x5FbDB2315678afecb367f032d93F642f64180aa3` :

```
cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 --rpc-url 127.0.0.1:8545
```

Example result (bytes32):

[illegible]

4. Decode the bytes32 value to a string:

[illegible]

Result: myPassword

The password is readable without calling `getPassword` and without being the owner.

RECOMMENDED MITIGATION

A password cannot be kept secret if it is stored in plaintext on-chain. The architecture needs to change.

One option is to keep the password off-chain, encrypt it, and store only the ciphertext (or a hash) on-chain. That design would require the owner to remember a separate secret used for decryption.

If the password is encrypted off-chain, consider removing the on-chain view function as well. An owner who later calls a decrypt-and-return helper from a public node or explorer could still leak the plaintext.

[H-2] Missing access control on `setPassword` allows anyone to overwrite the stored password

DESCRIPTION

`PasswordStore::setPassword` has no owner check. The NatSpec and protocol design say that only the owner should be able to store or update the password, but the function is `external` and accepts a call from any address.

```
function setPassword(string memory newPassword) external {
    // @audit no access control; any caller can overwrite s_password
    s_password = newPassword;
    emit SetNetPassword();
}
```

`getPassword` correctly reverts for non-owners with `PasswordStore__NotOwner`, but `setPassword` never performs that check.

IMPACT

Any address can overwrite the stored password. The owner can lose control of the value they intended to keep, and the contract no longer behaves as a single-user password store.

PROOF OF CONCEPT

Add the following test to `PasswordStore.t.sol`:

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);
    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

The test passes: a non-owner sets the password, and the owner later reads that attacker-controlled value.

RECOMMENDED MITIGATION

Enforce the same owner check used in `getPassword` :

```
if (msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

A dedicated `onlyOwner` modifier is also acceptable if the team prefers that style.

[L-1] The constructor does not set an initial password, so the store is empty until the first `setPassword`

DESCRIPTION

`PasswordStore` sets `s_owner` in the constructor but never sets `s_password` . Until someone calls `setPassword` , the stored value is the empty string.

```
constructor() {
    s_owner = msg.sender;
}
```

That gap matters more because of H-2: any address can make the first write. Even after an owner-set password, on-chain data remains public, as described in H-1.

IMPACT

Between deployment and the first `setPassword` , the store has no owner-chosen secret. If a non-owner writes first, the owner later reads an attacker-controlled value.

PROOF OF CONCEPT

Deploy the contract and call `getPassword` as the owner before any `setPassword` . The return value is empty.

RECOMMENDED MITIGATION

Accept an initial password in the constructor (or a one-time initializer restricted to the owner) so the store is never empty after deployment. This does not replace the access-control and on-chain-secrecy fixes.

[I-1] Incorrect NatSpec on `getPassword` documents a parameter the function does not accept

DESCRIPTION

The comment above `PasswordStore::getPassword` includes `@param newPassword` The new password to set . `getPassword` takes no parameters and does not set a password. The documentation incorrectly implies a signature such as `getPassword(string)` .

```
/*  
 * @notice This allows only the owner to retrieve the password.  
 * @param newPassword The new password to set.  
 */  
function getPassword() external view returns (string memory) {
```

IMPACT

Developers, auditors, and generated docs will be misled about the function's interface and purpose. This is a documentation error, not a runtime vulnerability.

RECOMMENDED MITIGATION

Remove the incorrect `@param` line.

```
- * @param newPassword The new password to set.
```